

```
"""
```

```
Simulation-only cross-DEX arbitrage scanner (Uniswap V2 vs SushiSwap, WETH/USDC).
```

```
It NEVER sends transactions and needs no private key. Each poll it:
```

1. reads pool reserves,
2. finds the input size that maximizes gross profit (ternary search),
3. subtracts an estimated gas cost,
4. logs the result to results.csv.

```
Usage:
```

```
export RPC_URL="https://your-ethereum-rpc"  
python scanner.py --polls 1000 --interval 12
```

```
"""
```

```
import argparse
```

```
import csv
```

```
import os
```

```
import time
```

```
from datetime import datetime, timezone
```

```
from web3 import Web3
```

```
# Verify these addresses on Etherscan before running.
```

```
UNI_PAIR = "0xB4e16d0168e52d35CaCD2c6185b44281Ec28C9Dc" # Uniswap V2 USDC/WETH
```

```
SUSHI_PAIR = "0x397FF1542f962076d0BFE58eA045FfA2d347ACa0" # SushiSwap USDC/WETH
```

```
# In both pairs token0 = USDC (6 decimals), token1 = WETH (18 decimals).
```

```
PAIR_ABI = [{
```

```
    "name": "getReserves", "type": "function", "stateMutability": "view",
```

```
    "inputs": [],
```

```
    "outputs": [{"name": "r0", "type": "uint112"},
```

```
                {"name": "r1", "type": "uint112"},
```

```
                {"name": "ts", "type": "uint32"}],
```

```
}]
```

```
FEE_NUM, FEE_DEN = 997, 1000 # 0.3% fee on both venues
```

```
GAS_UNITS = 250_000 # rough estimate for a two-swap arb; tune this
```

```
def amount_out(amt_in, r_in, r_out):
```

```
    """Uniswap V2 constant-product output ( $x*y=k$  with fee)."""
```

```
    if amt_in <= 0:
```

```
        return 0.0
```

```
    a = amt_in * FEE_NUM
```

```

    return a * r_out / (r_in * FEE_DEN + a)

def round_trip(usdc_in, buy_pool, sell_pool):
    """Buy WETH with USDC on buy_pool, sell it on sell_pool. Pools = (usdc_res, w
    weth = amount_out(usdc_in, buy_pool[0], buy_pool[1])
    return amount_out(weth, sell_pool[1], sell_pool[0])

def best_size(buy_pool, sell_pool, hi):
    """Ternary search: profit is concave in trade size."""
    lo = 0.0
    for _ in range(100):
        m1, m2 = lo + (hi - lo) / 3, hi - (hi - lo) / 3
        if round_trip(m1, buy_pool, sell_pool) - m1 < round_trip(m2, buy_pool, se
            lo = m1
        else:
            hi = m2
    size = (lo + hi) / 2
    return size, round_trip(size, buy_pool, sell_pool) - size

def get_pool(w3, addr):
    c = w3.eth.contract(address=Web3.to_checksum_address(addr), abi=PAIR_ABI)
    r0, r1, _ = c.functions.getReserves().call()
    return r0 / 1e6, r1 / 1e18 # (USDC, WETH)

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--polls", type=int, default=100)
    p.add_argument("--interval", type=float, default=12.0)
    p.add_argument("--out", default="results.csv")
    args = p.parse_args()

    w3 = Web3(Web3.HTTPProvider(os.environ["RPC_URL"]))
    new_file = not os.path.exists(args.out)
    with open(args.out, "a", newline="") as f:
        w = csv.writer(f)
        if new_file:
            w.writerow(["utc", "block", "uni_price", "sushi_price", "direction",
                        "best_size_usdc", "gross_usdc", "gas_usdc", "net_usdc"])
        for _ in range(args.polls):
            try:
                block = w3.eth.block_number
                uni, sushi = get_pool(w3, UNI_PAIR), get_pool(w3, SUSHI_PAIR)

```

```

p_uni, p_sushi = uni[0] / uni[1], sushi[0] / sushi[1] # USDC per
# Buy WETH where it is cheaper, sell where it is pricier.
if p_uni < p_sushi:
    direction, buy, sell = "uni->sushi", uni, sushi
else:
    direction, buy, sell = "sushi->uni", sushi, uni
size, gross = best_size(buy, sell, hi=buy[0] * 0.2)
gas_usdc = GAS_UNITS * w3.eth.gas_price / 1e18 * max(p_uni, p_sus
net = gross - gas_usdc
w.writerow([datetime.now(timezone.utc).isoformat(), block,
            round(p_uni, 2), round(p_sushi, 2), direction,
            round(size, 2), round(gross, 4), round(gas_usdc, 4),
            f.flush()
    print(f"block {block} {direction} size ${size:,.0f} gross ${gross
except Exception as e: # keep the run alive on RPC hiccups
    print("error:", e)
time.sleep(args.interval)

```

```

if __name__ == "__main__":
    main()

```